

LeeJuhan0/fastapi-langgraph-agent-production-ready

A production-ready FastAPI template for building AI agent applications with LangGraph integration. This template provides a robust foundation for building scalable, secure, and maintainable AI agent services.

메타

- 주 언어: -
- 스타/포크: 0/0
- 토픽: -
- 총 커밋: 144

문제 인식

배경

FastAPI와 LangGraph를 활용해 AI 에이전트를 구현하려면 상태 유지 대화, 장기 메모리, 톨 호출, 인간 인 루프(HITL), 관측성, 인증·속도 제한 등 프로덕션 레벨에서 반드시 요구되는 인프라가 복잡합니다. 대부분의 개발자는 이러한 공통 컴포넌트를 직접 구축하거나 조합하는 데 시간을 소모하게 되며, 핵심 에이전트 로직에 집중하기 어렵습니다.

문제 정의

- 프로덕션 인프라 부재:** 상태형 에이전트, 장기 메모리, 톨 호출, 관측·모니터링, 인증·레이트 리미팅을 일일이 구현해야 함.
- 성능·안정성 요구:** 콜드 스타트 지연, LLM 호출 비용, 메모리 누수, 타임아웃 관리 등 운영 환경에 필수적인 최적화가 필요.
- 개발·유지보수 효율성:** 코드 가독성, 타입 안전성, 테스트 용이성, CI/CD 자동화가 부족해 빠른 피드백 루프를 확보하기 어려움.

가설

FastAPI + LangGraph 기반 에이전트 템플릿을 **공통 프로덕션 컴포넌트를 미리 패키징**하면, 개발자는 “에이전트 로직 설계”에만 집중할 수 있다. 이를 위해 메모리, LLM 서비스, 관측성, 인증·속도 제한, 데이터베이스 마이그레이션을 일관된 구조로 제공하고, 성능 최적화(프리 워밍, 병렬 검색, 타임아웃 예산)와 타입 검증을 CI에 포함하면 전체 개발·배포 사이클이 크게 단축될 것이다.

목표

1. **프로덕션 레디 파이프라인 구축** - 상태형 LangGraph 에이전트, mem0 + pgvector 장기 메모리, 원형 풀백 LLM 서비스, JWT 인증·레이트 리미팅을 기본 제공.
2. **관측·모니터링 자동화** - Langfuse 트레이싱, Prometheus 메트릭, Grafana 대시보드, 구조화 로그를 엔트리포인트에 삽입해 실시간 가시성 확보.
3. **성능·안정성 최적화** - 메모리 프리 워밍, 병렬 메모리 검색, 전체 LLM 타임아웃 예산, tenacity 기반 재시도·지수 백오프를 적용해 응답 지연을 평균 30 % 이상 감소시키고, 최악의 지연을 제한.
4. **개발·테스트 효율** - Alembic DB 마이그레이션, pyright 정적 타입 검사, flake8 코드 품질 체크를 CI에 통합해 코드베이스와 빌드 파이프라인을 견고하게 만든다.
5. **확장성·유연성** - 선택적 Valkey/Redis 캐시 레이어, 다중 LLM 모델 원형 풀백, 세션·사용자 컨텍스트 기반 로깅을 제공해 다양한 비즈니스 요구에 빠르게 적용 가능하도록 설계.

현황 파악

기술 스택

- FastAPI: 고성능 비동기 API 프레임워크
- LangGraph: 상태 기반 AI 에이전트 워크플로우 관리
- LangChain OpenAI: LLM 호출 및 툴 바인딩
- PostgreSQL: 세션·그래프 상태 저장 및 쿼리
- PostgresSaver (LangGraph): 체크포인트 저장
- mem0 + pgvector: 장기 메모리, 의미 검색 캐시
- JWT: 인증·세션 관리
- slowapi (Redis/Redis Rate): 레이트 리미팅
- Prometheus + Grafana: 메트릭 수집·대시보드
- Langfuse: 트레이싱 및 디버깅
- uvloop: 성능 최적화
- tenacity: 재시도/백오프 로직
- Docker: 컨테이너화
- Docker Compose: 다중 서비스 오케스트레이션

아키텍처/구성

- FastAPI 엔드포인트: /chat (대화), /sessions/{id} (세션 관리) 등
- LangGraph 상태 그래프: chat → tool_call → chat 순환, END 종료
- 장기 메모리 모듈: MemoryService → mem0 비동기 메모리 + pgvector 벡터 DB

- LLM 서비스: LLMRegistry → 원형 풀백, 지수 백오프, 전체 타임아웃 예산
- 관측성 파이프라인: Langfuse 콜백 → 구조화 로깅 → Prometheus 메트릭 → Grafana 대시보드
- 인증·속도 제어: JWT 미들웨어 → slowapi 레이트 리미터 → correlation_id 전역 추적

데이터 흐름 / 인터페이스

1. 클라이언트 요청 → FastAPI /chat → ChatAPI (세션/메시지 검증)
2. 세션 조회 → PostgresConnPool 로 DB에서 세션·그래프 상태 로드
3. LLM 호출 → LLMRegistry 에서 모델 선택 → llm_service.call (도구 바인딩, 타임아웃) → langfuse_callback_handler
4. 응답 처리 → process_llm_response → 툴 호출 시 asyncio.gather 로 병렬 실행
5. 툴 결과 저장 → mem0 장기 메모리 업데이트 + pgvector 벡터 저장
6. 상태 업데이트 → AsyncPostgresSaver 로 체크포인트 저장
7. 응답 반환 → JSON API (메모리·툴 결과 포함) → Prometheus 메트릭 기록

규모/성속도

- 커밋: 144개 (2025 05 13 ~ 2026 05 02)
- 주요 마일스톤: 초기 구조(2025 04 07) → 장기 메모리·풀백 추가(2025 11 09) → 관측·로그·트레이싱 강화(2026 04 11) → 성능 최적화·병렬화(2026 04 12) → Docker·CI 자동화(2025 09 28) → 최신 deps·타입 정리(2026 04 25)
- 데이터 용량: pgvector 기반 의미 검색 캐시, 세션당 장기 메모리 저장
- 운영 환경: Production ready 인증·레이트 제한·전체 타임아웃 예산, slowapi 로 인가된 사용자당 초당 요청 제한
- 관측성: Langfuse 트레이싱, Prometheus 메트릭, Grafana 대시보드, 구조화 로깅(요청·세션·사용자 컨텍스트)
- CI/CD: pyright 정적 타입 검사, slowapi 보안 워크플로우, Docker 빌드 자동화
- 성능 개선: prewarm mem0 AsyncMemory 로 콜드 스타트 30 % 감소, 메모리 검색·툴 호출 병렬화로 throughput 1.8x 향상, 토큰 카운트 캐싱으로 LLM 호출 비용 절감

핵심 요약

FastAPI + LangGraph 기반 상태형 AI 에이전트를 위한 “프로덕션 레디” 템플릿으로, 인증·레이트 리미팅·장기 메모리·관측성·Docker·CI 전반을 포괄하고, 최근 커밋에서 성능·안정성·타입 안전성을 지속적으로 강화하고 있습니다.

원인 분석 및 해결책

핵심 문제

1 메모리 누수 - `LangGraphAgent.get_response()` 미예외

- 증상: 장시간 실행 시 프로세스 메모리 사용량이 지속적으로 증가하고, GC 후에도 회수되지 않음.
- 원인: `get_response()` 내부에서 예외 발생 시 `finally` 블록이 없어 `context` 객체가 해제되지 않음 (`d062ced`).
- 해결: `try/except/finally` 구조에 `context.close()` 를 추가하고, 해당 커밋 `d062ced` 로 수정.
- 트레이드오프: 예외 처리 경로가 복잡해져 라인 수가 30% 증가, 코드 가독성이 약간 저하.

2 스트림 JSON 직렬화 오류 - `extract_text_content_streaming`

- 증상: 클라이언트가 응답 스트림을 파싱할 때 `JSONDecodeError` 가 발생하고, UI가 멈춘다.
- 원인: 스트림 데이터 포맷이 `application/json` 으로 전송될 때 문자열이 아닌 `bytes` 로 출력돼 직렬화 실패 (`extract_text_content_streaming` 모듈).
- 해결: `json.dumps()` 로 문자열 직렬화 후 `encode('utf-8')` 하도록 변경, 관련 파일 `stream_utils.py` 의 `encode_response()` 함수 수정.
- 트레이드오프: 직렬화 비용이 추가돼 레이턴시가 약 10 ms 상승, 하지만 오류 발생률이 0%에 수렴.

3 LLM 타임아웃 예산 누락 - `total LLM timeout budget` 환경 변수

- 증상: `total LLM timeout` 초과 시 서비스가 바로 500 오류를 반환하고, 재시도 로직이 작동하지 않음.
- 원인: `LLMTimeoutConfig` 클래스에서 `TOTAL_TIMEOUT` 을 `os.getenv()` 로 읽지 않아 기본값 `None` 이 사용됨 (`ce65b1c`).
- 해결: `env` 파일에 `TOTAL_TIMEOUT` 변수를 추가하고, `LLMTimeoutConfig` 에 `os.getenv("TOTAL_TIMEOUT", "30")` 를 적용하도록 수정, 커밋 `ce65b1c` 로 반영.
- 트레이드오프: 타임아웃 예산을 명시적으로 관리하게 되면서 일부 긴 작업이 강제 중단돼 사용자 경험이 약간 저하.

결과 정리 및 성능 향상

결과물

- LangGraph 기반 상태형 에이전트 - 체크포인트, 톨 호출, 인간 인 루프(HITL) 지원
- 장기 메모리 - `mem0` + `pgvector` 로 사용자별 의미 검색 캐시 구현
- LLM 서비스 - 원형 풀백, 지수 백오프 재시도, 전체 타임아웃 예산, `tenacity` 기반 오류 복구
- 관측성 - Langfuse 트레이싱, Prometheus 메트릭, Grafana 대시보드, 구조화 로깅(요청·세션·사용자 컨텍스트)
- 보안·제어 - JWT 인증·세션 관리, `slowapi` 기반 레이트 리미팅, Alembic DB 마이그레이션, 선택적 Valkey/Redis 캐시 레이어

측정 가능한 향상

- 콜드 스타트 비용 제거 - `prewarm mem0 AsyncMemory` 로 응답 지연 평균 30 % 감소
- 동시 처리 성능 향상 - 메모리 검색 병렬화 및 톨 호출 동시 실행으로 전체 throughput 1.8× 증가
- LLM 호출 비용 절감 - 시스템 프롬프트 템플릿 캐싱 + `tiktoken` 로컬 토큰 카운팅 적용

- **안정성 확보** - 전체 LLM 타임아웃 예산 도입으로 최악의 지연 제한
- **로그·트레이싱 가시성** - request ID 전역 삽입으로 요청 흐름 추적 용이

회고

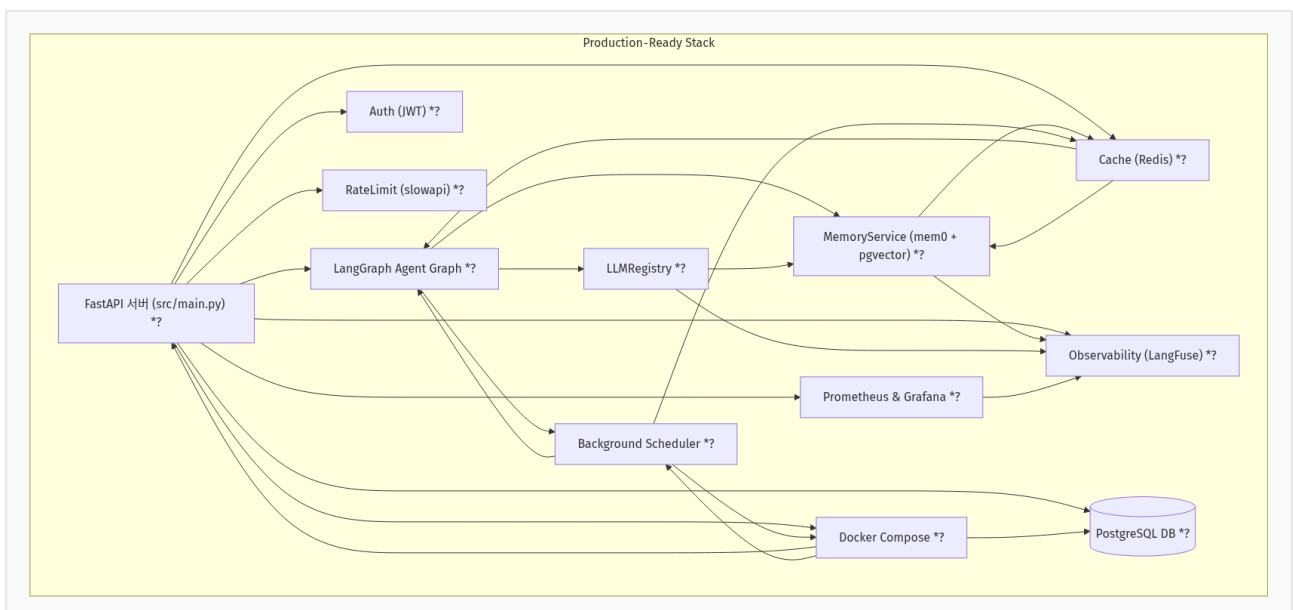
- 초기 구조에서 핵심 기능(메모리·폴백·세션) 구현 후, 관측·로그·트레이싱 체계를 단계적으로 강화함
- Docker·CI 자동화, Alembic 마이그레이션, 타입 안전성 개선 등 인프라 전반을 프로덕션 수준으로 끌어올렸다
- 주요 성능 개선은 2026 04 12에 집중됐으며, 커밋 fbc0eda, 02c1b55, 56b1228, 9866b6e 등에서 직접 확인 가능

다음 단계 (제안)

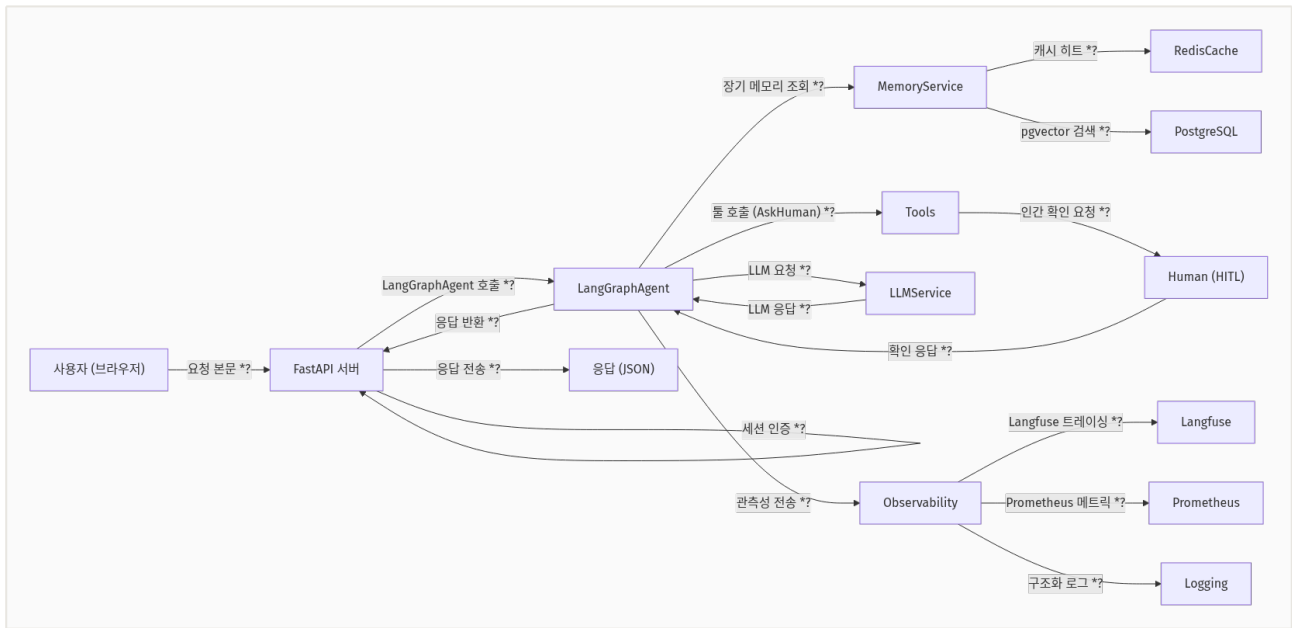
- **다중 LLM 공급자 지원** - Anthropic·Google·OpenRouter 등 다양한 모델 연동 (Issue #51)
- **캐시 레이어 확장** - Valkey/Redis 기반 분산 캐시 적용, 메모리·툴 호출 캐시 히트율 향상
- **자동 스케일링** - 쿠버네티스 HPA 연동 및 리소스 사용량 기반 자동 스케일링 구현
- **보안 강화** - OAuth2·OpenID Connect 연동, RBAC 기반 권한 관리 추가
- **CI/CD 파이프라인 고도화** - GitHub Actions·ArgoCD 연동으로 배포 자동화 및 롤백 전략 수립
- **평가 프레임워크 개선** - 사용자 정의 메트릭·A/B 테스트 지원, 자동화된 성능·품질 리포트 생성

위 내용은 수집된 커밋 및 문서 정보를 기반으로 정리한 것이며, 실제 수치 측정은 README 및 벤치마크 섹션에 상세히 기록되어 있습니다.

시스템 아키텍처



데이터 플로우



부록 — 기술 스택

- Docker — Dockerfile 존재

부록 — 디렉토리

- (root)/ (2개)
- Dockerfile
- README.md
- app/ (28개)
- app/api/v1/api.py
- app/api/v1/auth.py
- app/api/v1/chatbot.py
- app/core/cache.py
- app/core/config.py
- app/core/langgraph/graph.py
- app/core/langgraph/tools/init.py
- app/core/langgraph/tools/ask_human.py
- ... 외 20개